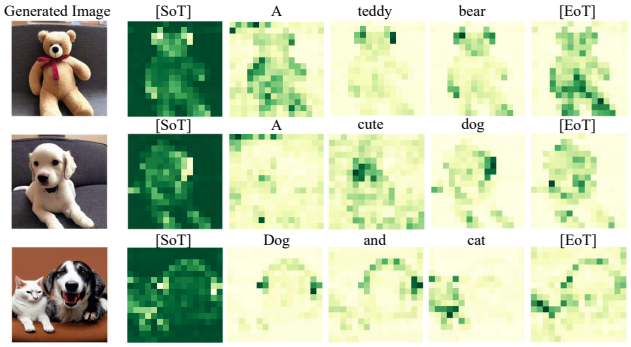


Figure 3: Cross-attention maps of different text prompts at the generation process. All of the examples indicate that start token ([SoT]) and padding tokens ([EoT]) carry rich semantic and layout information.

There is one more aspect of the SD architecture design that we require for our purposes. While there are several important design choices that make the network D work well in practice, the mechanism that is of interest in our investigation is *cross-attention*, which allows the generation process to be conditioned on text, thus connecting visual and textual information. Each cross-attention layer takes an intermediate feature tensor $\boldsymbol{z}^{(\gamma)} \in \mathbb{R}^{C \times \frac{H}{r} \times \frac{W}{r}}$ as input, where γ is the index of the relevant layer in the network, and r is a scaling factor defining the spatial resolution at that level of the representation. The cross-attention map $A^{(\gamma)}$ associates each spatial location $u \in \{1, \dots, \frac{H}{r}\} \times \{1, \dots, \frac{W}{r}\}$ to a token indexed by $i \in \{1, \dots, N\}$:

$$A_{ui}^{(\gamma)} = \frac{\exp\langle \boldsymbol{z}_u^{(\gamma)}, K_i^{(\gamma)} \rangle}{\sum_{j=1}^{N} \exp\langle \boldsymbol{z}_u^{(\gamma)}, K_j^{(\gamma)} \rangle}, \quad \boldsymbol{a}_u^{(\gamma)} = \sum_{i=1}^{N} A_{ui}^{(\gamma)} V_i^{(\gamma)},$$

where the value $V_i^{(\gamma)}$ and the key $K_i^{(\gamma)}$ are linear transformations of the token embedding Y_i provided by the textual encoder, and $\boldsymbol{a}_u^{(\gamma)}$ is the output of the cross-attention layer.

3.2. Layout Guidance

Next, we introduce our method for training-free layout guidance. It has already been shown that cross-attention layers regulate the spatial layout of the generated image [12]. Specifically, the attention map the attention map $A_{ui}^{(\gamma)}$ determines how strongly each location u in layer γ is associated with each of the N text tokens y_i. Since the sum of association strengths $\sum_{i=1}^{N} A_{ui}^{(\gamma)} = 1$ for each spatial location u, the different tokens can be thought of as "competing" for association with that location. To control the image layout using a bounding box B that corresponds to token y_i, the attention can be biased such that locations $u \in B$ within the target box are strongly associated with the token y_i (while other locations are not). We introduce two ways of

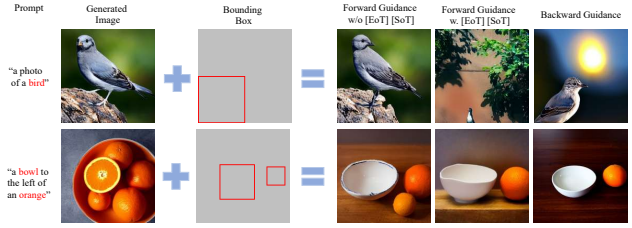


Figure 4: Comparison between forward and backward guidance, including guidance of start and padding tokens.

achieving this effect, which we refer to as forward guidance (Section 3.2.1) and backward guidance (Section 3.2.2). We give an overview of the two strategies in Figure 2.

3.2.1 Forward Guidance

In forward guidance, we represent the bounding box B as a smooth windowing function $g_u^{(\gamma)}$ which is equal to a constant $c > 0$ inside the box and quickly falls to zero outside.[1] We rescale the windowing function to sum to one ($\|g^{(\gamma)}\|_1 = 1$). Then, we bias the attention map $A^{(\gamma)}$ by replacing it via:

$$A_{ui}^{(\gamma)} \leftarrow (1 - \lambda) A_{ui}^{(\gamma)} + \lambda g_u^{(\gamma)} \sum_v A_{vi}^{(\gamma)}, \tag{1}$$

where $\lambda \in [0, 1]$ defines the strength of the intervention. Note that: (1) only the cross-attention map $A_{ui}^{(\gamma)}$ corresponding to token y_i is affected by this intervention (2) the window is weighed by the mass $\sum_v A_{vi}^{(\gamma)}$ so as to leave the latter unchanged.

This intervention is injected at each application of the denoiser function D at a selected subset of layers $\gamma \in \Gamma$. This means that the activation maps computed by each of the selected layers are replaced with Equation (1).

3.2.2 Backward Guidance

The second approach we consider is backward guidance. In this case, we bias the attention $A_{ui}^{(\gamma)}$ by introducing the energy function

$$E(A^{(\gamma)}, B, i) = \left(1 - \frac{\sum_{u \in B} A_{ui}^{(\gamma)}}{\sum_u A_{ui}^{(\gamma)}}\right)^2 \tag{2}$$

This is used in a manner similar to *classifier guidance* [3]. Specifically, at each application of the denoiser D, when layer $\gamma \in \Gamma$ is evaluated, the gradient of the loss (2) is computed via backpropagation to update the latent $\boldsymbol{z}_t (\equiv \boldsymbol{z}_t^{(0)})$:

$$\boldsymbol{z}_t \leftarrow \boldsymbol{z}_t - \sigma_t \eta \nabla_{\boldsymbol{z}_t} \sum_{\gamma \in \Gamma} E(A^{(\gamma)}, B, i), \tag{3}$$

[1] For simplicity, in our implementation, we put a Gaussian blob with σ decided by the resolution, height, and width of the bounding box.